

Pruebas

Simulador de Cola de Impresión FIFO

Nuevo Trabajo de Impresión

Usuario:

Ej. Juan Pérez

Nombre del Documento:

Ej. reporte_final.pdf

Número de Páginas:

1

Agregar a la Cola

Estado y Procesamiento

Trabajos pendientes en cola: 1

Siguiente en la cola:

reporte.pdf (por ana)

Imprimir Siguiente

Cola de Impresión Activa (Pendientes)

1. [ana] reporte.pdf (4 págs.)

Nuevo Trabajo de Impresión

Usuario:

Ej. Juan Pérez

Nombre del Documento:

Ej. reporte_final.pdf

Número de Páginas:

1

Agregar a la Cola

Estado y Procesamiento

Trabajos pendientes en cola: 3

Siguiente en la cola:

reporte.pdf (por ana)

Imprimir Siguiente

Cola de Impresión Activa (Pendientes)

1. [ana] reporte.pdf (4 págs.)

2. [Luis] Tarea.docx (2 págs.)

3. [Carlos] Grafica.png (1 págs.)

Nuevo Trabajo de Impresión

Usuario:

Ej. Juan Pérez

Nombre del Documento:

Ej. reporte_final.pdf

Número de Páginas:

1

Agregar a la Cola

Estado y Procesamiento

Trabajos pendientes en cola: 2

Siguiente en la cola:

Tarea.docx (por Luis)

Imprimir Siguiente

Cola de Impresión Activa (Pendientes)

1. [Luis] Tarea.docx (2 págs.)

2. [Carlos] Grafica.png (1 págs.)

Nuevo Trabajo de Impresión

Usuario:

Ej. Juan Pérez

Nombre del Documento:

Ej. reporte_final.pdf

Número de Páginas:

1

Agregar a la Cola

Estado y Procesamiento

Trabajos pendientes en cola: 2

Siguiente en la cola:

Grafica.png (por Carlos)

Imprimir Siguiente

Cola de Impresión Activa (Pendientes)

1. [Carlos] Grafica.png (1 págs.)

2. [Carlos] Manual.pdf (8 págs.)

Nuevo Trabajo de Impresión

Usuario:

Ej. Juan Pérez

Nombre del Documento:

Ej. reporte_final.pdf

Número de Páginas:

1

Agregar a la Cola

Estado y Procesamiento

Trabajos pendientes en cola: 0

Siguiente en la cola: Ninguno

Imprimir Siguiente

Cola de Impresión Activa (Pendientes)

Nuevo Trabajo de Impresión

Usuario:

juan

Nombre del Documento:

Ej. reporte_final.pdf

Número de Páginas:

1

Agregar a la Cola

Estado y Procesamiento

Trabajos pendientes en cola: 0

Siguiente en la cola: Ninguno

Imprimir Siguiente

Cola de Impresión Activa (Pendientes)

Cola Vacía

La cola de impresión está vacía.

OK

Prompts

Trabajo con pyqt5, hazme una interfaz en QTDESIGNER para un proyecto de un sistema FIFO que tenga los siguientes elementos: Entrada: usuario, nombre del documento y número de páginas.

- Encolar: al presionar el botón Agregar, crear un objeto TrabajoImpresion y colocarlo al final de la Queue.
- Procesar: al presionar Imprimir siguiente, retirar el elemento del frente y mostrar qué documento fue atendido.
- Consulta: mostrar en la interfaz todos los trabajos pendientes en su orden actual.
- Estado: mostrar cantidad de trabajos pendientes y, cuando sea posible, el elemento que está al frente.
- Validación mínima: no aceptar campos vacíos ni cantidades de páginas menores que 1.
- Restricción: no sustituir la clase Queue por una lista usada directamente desde la interfaz. Quiero que la interfaz sea de color negro con todos los widgets mas claros para que se note la diferencia, generame el codigo y que sea profesional.

```
<?xml version="1.0" encoding="UTF-8"?>

<ui version="4.0">

<class>VentanaColaImpresion</class>

<widget class="QDialog" name="VentanaColaImpresion">

<property name="geometry">

<rect>

<x>0</x>

<y>0</y>

<width>850</width>

<height>600</height>

</rect>

</property>

<property name="windowTitle">

<string>Simulador de Cola de Impresión FIFO</string>
```

```
</property>

<property name="styleSheet">

<string notr="true">

/* Fondo del Diálogo - Negro Puro */

QDialog {

    background-color: #0F0F0F;

}


/* Tarjetas contenedoras - Gris Oscuro */

QGroupBox {

    background-color: #1A1A1A;

    border: 1px solid #2B2B2B;

    border-radius: 8px;

    margin-top: 15px;

    font-size: 13px;

    font-weight: bold;

    color: #A0A0A0;

}

QGroupBox::title {

    subcontrol-origin: margin;

    subcontrol-position: top left;

    left: 15px;

    padding: 0 5px;

}


/* Textos generales */

QLabel {

    color: #D3D3D3;

    font-size: 12px;

}
```

```
/* Etiquetas de estado */
```

```
QLabel#lbl_total_pendientes {
```

```
    font-size: 13px;
```

```
    color: #888888;
```

```
}
```

```
QLabel#lbl_siguiete {
```

```
    font-size: 13px;
```

```
    font-weight: bold;
```

```
    color: #388E3C;
```

```
    background-color: #122214;
```

```
    padding: 8px;
```

```
    border-radius: 5px;
```

```
    border: 1px dashed #2E5C32;
```

```
}
```

```
/* Campos de entrada - Gris más claro */
```

```
QLineEdit, QSpinBox {
```

```
    background-color: #2D2D2D;
```

```
    border: 1px solid #3D3D3D;
```

```
    border-radius: 5px;
```

```
    color: #FFFFFF;
```

```
    padding: 6px 10px;
```

```
    font-size: 12px;
```

```
}
```

```
QLineEdit:focus, QSpinBox:focus {
```

```
    border: 1px solid #0084FF;
```

```
}
```

```
/* Lista de Consulta */
```

```
QListWidget {
```

```
    background-color: #1A1A1A;
```

```
border: 1px solid #2B2B2B;

border-radius: 8px;

color: #E0E0E0;

padding: 10px;

font-size: 13px;
}

QListWidget::item {

background-color: #242424;

border: 1px solid #2F2F2F;

border-radius: 5px;

padding: 8px;

margin-bottom: 6px;
}

/* Botones */

QPushButton {

background-color: #2D2D2D;

border: 1px solid #444444;

color: #FFFFFF;

border-radius: 5px;

padding: 8px;

font-weight: bold;

font-size: 12px;
}

QPushButton:hover {

background-color: #3A3A3A;

border-color: #555555;
}

QPushButton:pressed {

background-color: #242424;
}
```

```
/* Botón de Acción Especial (Imprimir) */
QPushButton#btn_imprimir {
    background-color: #0066CC;
    border: 1px solid #0052A3;
}
QPushButton#btn_imprimir:hover {
    background-color: #0080FF;
}
</string>
</property>
<layout class="QHBoxLayout" name="horizontalLayout">
    <property name="spacing">
        <number>20</number>
    </property>
    <property name="leftMargin">
        <number>25</number>
    </property>
    <property name="topMargin">
        <number>25</number>
    </property>
    <property name="rightMargin">
        <number>25</number>
    </property>
    <property name="bottomMargin">
        <number>25</number>
    </property>
    <item>
        <layout class="QVBoxLayout" name="leftPanel">
            <property name="spacing">
                <number>15</number>
```



```
</property>

<item>

<widget class="QGroupBox" name="group_input">

<property name="title">

<string>Nuevo Trabajo de Impresión</string>

</property>

<layout class="QVBoxLayout" name="verticalLayout_2">

<property name="spacing">

<number>10</number>

</property>

<item>

<widget class="QLabel" name="lbl_usuario_title">

<property name="text">

<string>Usuario:</string>

</property>

</widget>

</item>

<item>

<widget class="QLineEdit" name="txt_usuario">

<property name="placeholderText">

<string>Ej. Juan Pérez</string>

</property>

</widget>

</item>

<item>

<widget class="QLabel" name="lbl_doc_title">

<property name="text">

<string>Nombre del Documento:</string>

</property>

</widget>

</item>
```

```
<item>

<widget class="QLineEdit" name="txt_documento">

<property name="placeholderText">

<string>Ej. reporte_final.pdf</string>

</property>

</widget>

</item>

<item>

<widget class="QLabel" name="lbl_paginas_title">

<property name="text">

<string>Número de Páginas:</string>

</property>

</widget>

</item>

<item>

<widget class="QSpinBox" name="spin_paginas">

<property name="minimum">

<number>1</number>

</property>

<property name="maximum">

<number>9999</number>

</property>

</widget>

</item>

<item>

<widget class="QPushButton" name="btn_agregar">

<property name="text">

<string>Agregar a la Cola</string>

</property>

</widget>

</item>
```

```
</layout>

</widget>

</item>

<item>

<widget class="QGroupBox" name="group_status">

<property name="title">

<string>Estado y Procesamiento</string>

</property>

<layout class="QVBoxLayout" name="verticalLayout_3">

<property name="spacing">

<number>12</number>

</property>

<item>

<widget class="QLabel" name="lbl_total_pendientes">

<property name="text">

<string>Trabajos pendientes: 0</string>

</property>

</widget>

</item>

<item>

<widget class="QLabel" name="lbl_siguiente">

<property name="text">

<string>Siguiendo en la cola: Ninguno</string>

</property>

<property name="wordWrap">

<bool>true</bool>

</property>

</widget>

</item>

<item>

<widget class="QPushButton" name="btn_imprimir">
```

```
<property name="text">
    <string>Imprimir Siguiente</string>
</property>
</widget>
</item>
</layout>
</widget>
</item>
<item>
    <spacer name="verticalSpacer">
        <property name="orientation">
            <enum>Qt::Vertical</enum>
        </property>
        <property name="sizeHint" stdset="0">
            <size>
                <width>20</width>
                <height>40</height>
            </size>
        </property>
    </spacer>
</item>
</layout>
</item>
<item>
    <layout class="QVBoxLayout" name="rightPanel">
        <item>
            <widget class="QGroupBox" name="group_queue">
                <property name="title">
                    <string>Cola de Impresión Activa (Pendientes)</string>
                </property>
                <layout class="QVBoxLayout" name="verticalLayout_4">
```

```

<item>

    <widget class="QListWidget" name="list_visual"/>

</item>

</layout>

</widget>

</item>

</layout>

</item>

</layout>

</widget>

<resources/>

<connections/>

</ui>

```

todas las instrucciones que te di las ponlas en un controlador, haz las operaciones necesarias y cuando yo te indique hazme el archivo load para conectar la interfaz con las operaciones, pero ahorita solo dame el controlador. Esta es mi clase cola(queue) from estructuras.lineales.nodo import Node

```

class Queue(object):

    def __init__(self):

        self.first = None

        self.last = None

        self.size = 0

    def enqueue(self, valor):

        # Agrega el valor al final de la cola

        nuevo_nodo = Node(valor)

        if self.isEmpty():

            self.first = nuevo_nodo

            self.last = nuevo_nodo

        else:

            self.last.next = nuevo_nodo

```

```

        self.last = nuevo_nodo

    self.size += 1

def dequeue(self):
    # Elimina el primer elemento que entró
    if self.isEmpty():
        return None

    eliminado = self.first.data # <-- Corregido: .data en lugar de .value
    self.first = self.first.next

    if self.first is None:
        self.last = None

    self.size -= 1

    return eliminado

def firstQueue(self):
    # Consulta el primer elemento sin eliminarlo
    if self.isEmpty():
        return None

    return self.first.data # <-- Corregido: .data en lugar de .value

def lastQueue(self):
    # Consulta el último elemento sin eliminarlo
    if self.isEmpty():
        return None

    return self.last.data # <-- Corregido: .data en lugar de .value

def printQueue(self):
    # Devuelve el contenido completo de la cola

```

```

if self.isEmpty():
    return "Cola vacía"

elementos = []

actual = self.first

while actual is not None:
    elementos.append(str(actual.data)) # <-- Corregido: .data en lugar de .value
    actual = actual.next

return " -> ".join(elementos)

```

```

def isEmpty(self):
    return self.first == None

```

```

def to_list(self):
    """Convierte los valores de la cola en una lista común de Python"""

    elementos = []

    actual = self.first

    while actual is not None:
        elementos.append(actual.data) # <-- Corregido: .data en lugar de .value
        actual = actual.next

    return elementos

```

```

# controlador_fifo.py

```

```

# Importación de tu estructura de datos Queue

```

```

# (Ajusta la ruta de importación si tu archivo queue.py está en otra carpeta)

```

```

from estructuras.lineales.cola import Queue

```

```

# =====

```

```

# 1. MODELO DE DATOS

```

```
# =====
```

```
class TrabajoImpresion:
```

```
    """Representa cada documento encolado."""
```

```
    def __init__(self, usuario: str, documento: str, paginas: int):
```

```
        self.usuario = usuario
```

```
        self.documento = documento
```

```
        self.paginas = paginas
```

```
    def __str__(self):
```

```
        return f"[{self.usuario}] {self.documento} [{self.paginas} págs.]"
```

```
# =====
```

```
# 2. CONTROLADOR DE OPERACIONES FIFO
```

```
# =====
```

```
class ControladorFIFO:
```

```
    """
```

```
    Controlador que aísla la lógica de la interfaz.
```

```
    Maneja la cola de impresión y valida las restricciones del sistema.
```

```
    """
```

```
    def __init__(self):
```

```
        # Se inicializa tu clase Queue importada
```

```
        self._cola_impresion = Queue()
```

```
    def agregar_trabajo(self, usuario: str, documento: str, paginas: int) -> tuple[bool, str]:
```

```
        """
```

```
        Intenta encolar un nuevo trabajo de impresión.
```

```
        Retorna una tupla (éxito, mensaje_de_error_o_confirmacion).
```

```
        """
```



```

# Validación mínima: Campos vacíos

if not usuario.strip() or not documento.strip():

    return False, "Error: No se aceptan campos vacíos."


# Validación mínima: Páginas menores que 1

if paginas < 1:

    return False, "Error: El número de páginas debe ser mayor o igual a 1."


# Encolar elemento

nuevo_trabajo = TrabajoImpresion(usuario.strip(), documento.strip(), paginas)

self._cola_impresion.enqueue(nuevo_trabajo)

return True, f"Trabajo '{documento}' agregado con éxito."


def procesar_siguiente(self) -> tuple[TrabajoImpresion, str] | tuple[None, str]:
    """
    Retira y retorna el elemento al frente (FIFO).

    Retorna una tupla (TrabajoImpresion, mensaje) o (None, mensaje de cola vacía).
    """

    if self._cola_impresion.isEmpty():

        return None, "La cola de impresión está vacía."

    trabajo_atendido = self._cola_impresion.dequeue()

    return trabajo_atendido, f"Se ha impreso exitosamente: {trabajo_atendido.documento}"


def obtener_pendientes(self) -> list:
    """Retorna todos los trabajos actualmente encolados en su orden correspondiente."""

    return self._cola_impresion.to_list()


def obtener_siguiente_en_espera(self) -> TrabajoImpresion | None:
    """Consulta el elemento al frente de la cola sin retirarlo."""

    return self._cola_impresion.firstQueue()

```

```
def obtener_total_pendientes(self) -> int:

    """Retorna la cantidad actual de trabajos en la cola."""

    return self._cola_impresion.size
```

hazme el archivo load conectando los botones de la interfaz con cada una de las acciones que te pedi

Aquí tienes el archivo **main.py** (el cargador o loader). Este script se encarga de iniciar la aplicación de PyQt5, cargar dinámicamente tu archivo interfaz.ui, e intercomunicar cada botón y elemento visual con las funciones lógicas de tu ControladorFIFO.

Guarda este archivo como **main.py** en la misma carpeta donde tienes tu interfaz.ui y tu controlador_fifo.py:

Python

```
# main.py
```

```
import sys
```

```
from PyQt5.QtWidgets import QApplication, QMainWindow, QMessageBox
```

```
from PyQt5 import uic
```

```
# Importamos el controlador que creamos en el paso anterior
```

```
from controlador_fifo import ControladorFIFO
```

```
class VentanaColaImpresion(QMainWindow):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        # 1. Cargar la interfaz gráfica desde el archivo .ui de QtDesigner
```

```
        uic.loadUi("interfaz.ui", self)
```

```
        # 2. Instanciar nuestro controlador (el cerebro FIFO)
```

```
        self.controlador = ControladorFIFO()
```

```
        # 3. Conectar los eventos de los botones con los métodos de esta clase
```

```
        self.btn_agregar.clicked.connect(self.action_agregar)
```

```
self.btn_imprimir.clicked.connect(self.action_procesar)
```

```
# 4. Actualizar el estado visual inicial
```

```
self.actualizar_vista()
```

```
# =====
```

```
# MÉTODOS DE ACCIÓN (CONEXIÓN CON EL BOTÓN)
```

```
# =====
```

```
def action_agregar(self):
```

```
    """Acción al presionar el botón 'Agregar a la Cola.'"""
```

```
    # Obtener los datos directamente de los inputs de la interfaz
```

```
    usuario = self.txt_usuario.text()
```

```
    documento = self.txt_documento.text()
```

```
    paginas = self.spin_paginas.value()
```

```
    # Enviar los datos al controlador para que los valide y los encole
```

```
    exito, mensaje = self.controlador.agregar_trabajo(usuario, documento, paginas)
```

```
    if exito:
```

```
        # Si se agregó con éxito, limpiamos los campos para el siguiente ingreso
```

```
        self.txt_usuario.clear()
```

```
        self.txt_documento.clear()
```

```
        self.spin_paginas.setValue(1)
```

```
        # Enfocamos de nuevo el primer campo para comodidad del usuario
```

```
        self.txt_usuario.setFocus()
```

```
    # Actualizamos la pantalla
```

```
    self.actualizar_vista()
```

```
else:
```

```
    # Si falló la validación, mostramos la alerta de error en pantalla
```

```

        QMessageBox.warning(self, "Validación de Entrada", mensaje)

def action_procesar(self):
    """Acción al presionar el botón 'Imprimir Siguiente.'"""

    # Intentamos retirar el elemento del frente a través del controlador
    trabajo, mensaje = self.controlador.procesar_siguiente()

    if trabajo is not None:

        # Si había un trabajo, mostramos un mensaje de éxito detallando qué se atendió
        QMessageBox.information(
            self,
            "Procesando Documento",
            f"🖨 Se atendió exitosamente el siguiente trabajo:\n\n"
            f"• Documento: {trabajo.documento}\n"
            f"• Solicitado por: {trabajo.usuario}\n"
            f"• Páginas: {trabajo.paginas}"
        )

        # Actualizamos la pantalla para reflejar el cambio
        self.actualizar_vista()

    else:

        # Si la cola estaba vacía, el controlador avisa y mandamos una advertencia
        QMessageBox.information(self, "Cola Vacía", mensaje)

# =====
# MÉTODO DE ACTUALIZACIÓN VISUAL (SINCRO)
# =====

def actualizar_vista(self):
    """Sincroniza todos los widgets de la interfaz con los datos del controlador."""

```

1. ACTUALIZAR CONSULTA: Mostrar todos los trabajos pendientes en su orden actual

```
self.list_visual.clear()
```

```
trabajos_pendientes = self.controlador.obtener_pendientes()
```

```
for i, trabajo in enumerate(trabajos_pendientes, start=1):
```

```
    # Agregamos cada objeto de impresión al QListWidget usando su método __str__
```

```
    self.list_visual.addItem(f"{i}. {trabajo}")
```

2. ACTUALIZAR ESTADO: Cantidad de trabajos pendientes

```
total = self.controlador.obtener_total_pendientes()
```

```
self.lbl_total_pendientes.setText(f"Trabajos pendientes en cola: {total}")
```

3. ACTUALIZAR ESTADO: Elemento que está al frente (Peek) sin retirarlo

```
siguiente = self.controlador.obtener_siguiente_en_espera()
```

```
if siguiente:
```

```
    self.lbl_siguiente.setText(
```

```
        f"Siguiente en la cola:\n 📄 {siguiente.documento} (por {siguiente.usuario})"
```

```
    )
```

```
else:
```

```
    self.lbl_siguiente.setText("Siguiente en la cola: Ninguno")
```

```
# =====
```

```
# INICIO DE LA APLICACIÓN
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    app = QApplication(sys.argv)
```

```
    ventana = VentanaColaImpresion()
```

```
    ventana.show()
```

```
    sys.exit(app.exec_())
```

